

# Code Change Report

Project Fix Agent | 11 file(s) modified

■ src/main/java/com/library/model/User.java

@@ -4,8 +4,8 @@

import lombok.Data;

import lombok.NoArgsConstructor;

-import javax.persistence.\*; // javax -> jakarta in Boot 3

-import javax.validation.constraints.\*; // javax -> jakarta in Boot 3

+import jakarta.persistence.\*;

+import jakarta.validation.constraints.\*;

import java.time.LocalDateTime;

import java.util.HashSet;

import java.util.Set;

@@ -10,47 +10,51 @@

import java.util.HashSet;

import java.util.Set;

-@Entity

-@Table(name = "users")

-@Data

-@NoArgsConstructor

-@AllArgsConstructor

-public class User {

+ @Entity

+ @Table(name = "users")

+ @Data

+ @NoArgsConstructor

+ @AllArgsConstructor

+ public class User {

- @Id

- @GeneratedValue(strategy = GenerationType.IDENTITY)

- private Long id;

+ @Id

+ @GeneratedValue(strategy = GenerationType.IDENTITY)

+ private Long id;

- @Column(unique = true, nullable = false)

- @NotBlank

- @Size(min = 3, max = 50)

- private String username;

+ @Column(unique = true, nullable = false)

+ @NotBlank

+ @Size(min = 3, max = 50)

```

+ private String username;

- @Column(unique = true, nullable = false)
- @NotBlank
- @Email
- private String email;
+ @Column(unique = true, nullable = false)
+ @NotBlank
+ @Email
+ private String email;

- @Column(nullable = false)
- @NotBlank
- @Size(min = 8)
- private String password;
+ @Column(nullable = false)
+ @NotBlank
+ @Size(min = 8)
+ private String password;

- @Column(name = "full_name")
- private String fullName;
+ @Column(name = "full_name")
+ private String fullName;

- @ElementCollection(fetch = FetchType.EAGER)
- @CollectionTable(name = "user_roles", joinColumns = @JoinColumn(name = "user_id"))
- @Column(name = "role")
- private Set<String> roles = new HashSet<>();
+ @ElementCollection(fetch = FetchType.EAGER)
+ @CollectionTable(name = "user_roles", joinColumns = @JoinColumn(name = "user_id"))
+ @Column(name = "role")
+ private Set<String> roles = new HashSet<>();

- @Column(name = "is_active")
- private Boolean active = true;
+ @Column(name = "is_active")
+ private Boolean active = true;

- @Column(name = "created_at")
- private LocalDateTime createdAt;
+ @Column(name = "created_at")
+ private LocalDateTime createdAt;

- @PrePersist
+ @PrePersist
+ public void prePersist() {
+ this.createdAt = LocalDateTime.now();
+ }
+ }

public void prePersist() {
this.createdAt = LocalDateTime.now();

```

```

}
@@ -55,7 +55,4 @@
    this.createdAt = LocalDateTime.now();
}
}
- public void prePersist() {
-     this.createdAt = LocalDateTime.now();
- }
}

```

```

@@ -54,5 +54,5 @@
public void prePersist() {
    this.createdAt = LocalDateTime.now();
}
- }
}

```

```

+}

```

```

@@ -55,4 +55,3 @@
    this.createdAt = LocalDateTime.now();
}
}
-}

```

■ src/main/java/com/library/dto/RegisterRequestDto.java

```

@@ -4,7 +4,7 @@
import lombok.Data;
import lombok.NoArgsConstructor;
- import javax.validation.constraints.*; // javax -> jakarta in Boot 3
+ import jakarta.validation.constraints.*;
@Data @NoArgsConstructor @AllArgsConstructor
public class RegisterRequestDto {

```

■ src/main/java/com/library/model/Book.java

```

@@ -10,8 +10,8 @@
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;
- import javax.persistence.*; // javax.persistence.* -> jakarta.persistence.* in Spring Boot 3
- import javax.validation.constraints.*; // javax.validation.* -> jakarta.validation.* in Spring Boot 3
+ import jakarta.persistence.*;
+ import jakarta.validation.constraints.*;
import java.time.LocalDateTime;
import java.util.ArrayList;

```

```

import java.util.List;

@@ -74,7 +74,7 @@
// Old Hibernate @Type annotation – API REMOVED/CHANGED in Hibernate 6
// In Hibernate 6: use @Convert(converter = YesNoConverter.class) or @JdbcTypeCode
- @Type(type = "yes_no") // ← COMPILER ERROR in Hibernate 6+: @Type(type=) no longer accepted
+ @Convert(converter = YesNoConverter.class)
@Column(name = "is_active", nullable = false)
private Boolean active = true;

@@ -74,7 +74,8 @@
// Old Hibernate @Type annotation – API REMOVED/CHANGED in Hibernate 6
// In Hibernate 6: use @Convert(converter = YesNoConverter.class) or @JdbcTypeCode
- @Convert(converter = YesNoConverter.class)
+ @Column(name = "is_active", nullable = false)
+ private Boolean active = true;
@Column(name = "is_active", nullable = false)
private Boolean active = true;

```

■ src/main/java/com/library/model/Tag.java

```

@@ -4,7 +4,7 @@
import lombok.Data;
import lombok.NoArgsConstructor;

-import javax.persistence.*; // javax -> jakarta in Boot 3
+import jakarta.persistence.*;
@Entity
@Table(name = "tags")

```

■ src/main/java/com/library/model/BorrowRecord.java

```

@@ -9,8 +9,9 @@
import java.time.LocalDate;
import java.time.LocalDateTime;

-@Entity
-@Table(name = "borrow_records")
+import jakarta.persistence.*;
+import java.time.LocalDate;
+import java.time.LocalDateTime;
@Data
@NoArgsConstructor
@AllArgsConstructor

@@ -5,7 +5,7 @@
import lombok.NoArgsConstructor;
import org.hibernate.annotations.Type; // Changed in Hibernate 6
-import javax.persistence.*; // javax -> jakarta in Boot 3

```

```

+import jakarta.persistence.*;
import java.time.LocalDate;
import java.time.LocalDateTime;
@@ -40,7 +40,7 @@
// Old Hibernate @Type – BREAKS in Hibernate 6
// Must use @Convert(converter = YesNoConverter.class) instead
- @Type(type = "yes_no") // ← COMPILER ERROR in Hibernate 6+
+ @Convert(converter = YesNoConverter.class)
@Column(name = "is_returned")
private Boolean returned = false;

```

■ src/main/java/com/library/dto/BookDto.java

```

@@ -36,6 +36,7 @@
@Min(0)
private Integer availableCopies;
+ private Integer availableCopies;
private Boolean active;
@@ -5,7 +5,7 @@
import lombok.Data;
import lombok.NoArgsConstructor;
- import javax.validation.constraints.*; // javax -> jakarta in Boot 3
+ import jakarta.validation.constraints.*;
import java.time.LocalDateTime;
@Data
@@ -36,7 +36,6 @@
@Min(0)
private Integer availableCopies;
- private Integer availableCopies;
private Boolean active;

```

■ src/main/java/com/library/exception/GlobalExceptionHandler.java

```

@@ -9,7 +9,7 @@
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
- import javax.servlet.http.HttpServletRequest; // javax -> jakarta in Boot 3
+ import jakarta.servlet.http.HttpServletRequest;
import java.util.HashMap;
import java.util.Map;
@@ -24,7 +24,7 @@
@ExceptionHandler(BookNotFoundException.class)
public ResponseEntity<ApiResponse<Void>> handleBookNotFound(
- BookNotFoundException ex, HttpServletRequest request) { // javax -> jakarta

```

```

+ BookNotFoundException ex, HttpServletRequest request) {
return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ApiResponse.error(ex.getMessage()));
}

@@ -30,7 +30,7 @@
ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<ApiResponse<Void>> handleUserNotFound(
- UserNotFoundException ex, HttpServletRequest request) {
+ UserNotFoundException ex, HttpServletRequest request) {
return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ApiResponse.error(ex.getMessage()));
}

@@ -48,7 +48,7 @@
ExceptionHandler(BookNotAvailableException.class)
public ResponseEntity<ApiResponse<Void>> handleBookNotAvailable(
- BookNotAvailableException ex, HttpServletRequest request) {
+ BookNotAvailableException ex, HttpServletRequest request) {
return
ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY).body(ApiResponse.error(ex.getMessage()));
}

@@ -70,7 +70,7 @@
}
ExceptionHandler(Exception.class)
- public ResponseEntity<ApiResponse<Void>> handleGeneral(Exception ex, HttpServletRequest request) {
+ public ResponseEntity<ApiResponse<Void>> handleGeneral(Exception ex, HttpServletRequest request) {
return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
.body(ApiResponse.error("Internal server error: " + ex.getMessage()));
}

```

■ src/main/java/com/library/security/JwtAuthenticationEntryPoint.java

```

@@ -7,8 +7,8 @@
import org.springframework.security.web.AuthenticationEntryPoint;
import org.springframework.stereotype.Component;

- import javax.servlet.http.HttpServletRequest;
- import javax.servlet.http.HttpServletResponse;
+ import jakarta.servlet.http.HttpServletRequest;
+ import jakarta.servlet.http.HttpServletResponse;

@Component
public class JwtAuthenticationEntryPoint implements AuthenticationEntryPoint, Serializable {

@@ -16,7 +16,7 @@
private static final long serialVersionUID = -7858869558953243875L;

@Override
- public void commence(HttpServletRequest request, // javax -> jakarta
+ public void commence(HttpServletRequest request,
HttpServletResponse response, // javax -> jakarta
AuthenticationException authException) throws IOException {

```

```
response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Unauthorized");
```

```
@@ -17,7 +17,7 @@
```

```
@Override
```

```
public void commence(HttpServletRequest request,
```

```
- HttpServletResponse response, // javax -> jakarta
```

```
+ HttpServletResponse response,
```

```
AuthenticationException authException) throws IOException {
```

```
response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Unauthorized");
```

```
}
```

```
@@ -19,6 +19,6 @@
```

```
public void commence(HttpServletRequest request,
```

```
HttpServletResponse response,
```

```
AuthenticationException authException) throws IOException {
```

```
- response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Unauthorized");
```

```
+ response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Unauthorized");
```

```
}
```

```
}
```

```
■ src/main/java/com/library/security/JwtRequestFilter.java
```

```
@@ -13,10 +13,10 @@
```

```
import com.library.service.UserDetailsServiceImpl;
```

```
import io.jsonwebtoken.ExpiredJwtException;
```

```
-import javax.servlet.FilterChain;
```

```
-import javax.servlet.ServletException;
```

```
-import javax.servlet.http.HttpServletRequest;
```

```
-import javax.servlet.http.HttpServletResponse;
```

```
+import jakarta.servlet.FilterChain;
```

```
+import jakarta.servlet.ServletException;
```

```
+import jakarta.servlet.http.HttpServletRequest;
```

```
+import jakarta.servlet.http.HttpServletResponse;
```

```
/**
```

```
 * JWT Request Filter
```

```
@@ -36,7 +36,7 @@
```

```
private JwtTokenUtil jwtTokenUtil;
```

```
@Override
```

```
- protected void doFilterInternal(HttpServletRequest request, // javax -> jakarta
```

```
+ protected void doFilterInternal(HttpServletRequest request,
```

```
HttpServletResponse response, // javax -> jakarta
```

```
FilterChain filterChain) // javax -> jakarta
```

```
throws ServletException, IOException {
```

```
@@ -37,7 +37,7 @@
```

```
@Override
```

```
protected void doFilterInternal(HttpServletRequest request,
```

```
- HttpServletResponse response, // javax -> jakarta
```

```

+ HttpServletResponse response,
FilterChain filterChain) // javax -> jakarta
throws ServletException, IOException {
@@ -38,7 +38,7 @@
@Override
protected void doFilterInternal(HttpServletRequest request,
HttpServletResponse response,
- FilterChain filterChain) // javax -> jakarta
+ FilterChain filterChain
throws ServletException, IOException {
final String requestTokenHeader = request.getHeader("Authorization");
@@ -38,7 +38,7 @@
@Override
protected void doFilterInternal(HttpServletRequest request,
HttpServletResponse response,
- FilterChain filterChain
+ FilterChain filterChain)
throws ServletException, IOException {
final String requestTokenHeader = request.getHeader("Authorization");

```

■ src/main/java/com/library/security/SecurityConfig.java

```

@@ -11,7 +11,7 @@
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.builders.WebSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
-import
org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerAdapter; //
REMOVED in Spring Security 5.7+ / 6.x
+import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
@@ -31,7 +31,7 @@
@Configuration
@EnableWebSecurity
@EnableGlobalMethodSecurity(prePostEnabled = true, securedEnabled = true) // Deprecated: use
@EnableMethodSecurity
-public class SecurityConfig extends WebSecurityConfigurerAdapter { // ← REMOVED in Spring Security 6
+public class SecurityConfig {
@Autowired
private UserDetailsServiceImpl userDetailsService;
@@ -53,7 +53,7 @@
@Override
public AuthenticationManager authenticationManagerBean() throws Exception {

```



```
// authenticationManagerBean() removed from WebSecurityConfigurerAdapter in Spring 6
```

```
- return super.authenticationManagerBean();
```

```
+ return authenticationManagerBuilder.getOrBuild();
```

```
}
```

```
@Override
```

```
@@ -60,7 +60,12 @@
```

```
public void configure(WebSecurity web) {
```

```
// WebSecurity.ignoring() is deprecated – use requestMatchers in HttpSecurity
```

```
web.ignoring()
```

```
- .antMatchers("/h2-console/**") // antMatchers() removed in Spring 6
```

```
+ web.ignoring()
```

```
+ .requestMatchers("/h2-console/**")
```

```
+ .requestMatchers("/swagger-ui.html")
```

```
+ .requestMatchers("/swagger-resources/**")
```

```
+ .requestMatchers("/v2/api-docs")
```

```
+ .requestMatchers("/webjars/**");
```

```
.antMatchers("/swagger-ui.html")
```

```
.antMatchers("/swagger-resources/**")
```

```
.antMatchers("/v2/api-docs")
```

```
@@ -67,7 +67,7 @@
```

```
.requestMatchers("/v2/api-docs")
```

```
.requestMatchers("/webjars/**");
```

```
.antMatchers("/swagger-ui.html")
```

```
- .antMatchers("/swagger-resources/**")
```

```
+ @Bean
```

```
.antMatchers("/v2/api-docs")
```

```
.antMatchers("/webjars/**");
```

```
}
```

```
@@ -78,7 +78,7 @@
```

```
.csrf().disable()
```

```
.exceptionHandling()
```

```
.authenticationEntryPoint(jwtAuthEntryPoint)
```

```
- .and()
```

```
+ .requestMatchers("/api/auth/**").permitAll()
```

```
.sessionManagement()
```

```
.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
```

```
.and()
```

```
@@ -59,14 +59,14 @@
```

```
@Override
```

```
public void configure(WebSecurity web) {
```

```
// WebSecurity.ignoring() is deprecated – use requestMatchers in HttpSecurity
```

```
- web.ignoring()
```

```
- web.ignoring()
```

```
- .requestMatchers("/h2-console/**")
```

```
- .requestMatchers("/swagger-ui.html")
```

```
- .requestMatchers("/swagger-resources/**")
```

```
- .requestMatchers("/v2/api-docs")
- .requestMatchers("/webjars/**");
- .antMatchers("/swagger-ui.html")
```

```
+ web.ignoring()
+ .requestMatchers(
+ "/h2-console/**",
+ "/swagger-ui.html",
+ "/swagger-resources/**",
+ "/v2/api-docs",
+ "/webjars/**"
+ );
```

@Bean

```
.antMatchers("/v2/api-docs")
.antMatchers("/webjars/**");
```

@@ -67,10 +67,29 @@

```
"/v2/api-docs",
"/webjars/**"
);
```

- @Bean

```
- .antMatchers("/v2/api-docs")
- .antMatchers("/webjars/**");
- }
```

+ @Bean

```
+ public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
+ http
+ .csrf().disable()
+ .exceptionHandling()
+ .authenticationEntryPoint(jwtAuthEntryPoint)
+ .and()
+ .sessionManagement()
+ .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
+ .and()
+ .authorizeHttpRequests()
+ .requestMatchers("/api/auth/**").permitAll()
+ .requestMatchers(HttpMethod.GET, "/api/books/**").permitAll()
+ .requestMatchers("/api/admin/**").hasRole("ADMIN")
+ .anyRequest().authenticated()
+ .and()
+ .headers()
+ .frameOptions().disable();
+
+ http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
+
+ return http.build();
+ }
```

@Override

```
protected void configure(HttpSecurity http) throws Exception {
```

```
@@ -62,11 +62,11 @@
```

```
web.ignoring()
```

```
.requestMatchers(
```

```
"/h2-console/**",
```

```
- "/swagger-ui.html",
```

```
- "/swagger-resources/**",
```

```
- "/v2/api-docs",
```

```
+ "/swagger-ui/**",
```

```
+ "/v3/api-docs/**",
```

```
"/webjars/**"
```

```
);
```

```
+ }
```

```
@Bean
```

```
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
```

```
http
```

```
@@ -83,10 +83,9 @@
```

```
.requestMatchers("/api/admin/**").hasRole("ADMIN")
```

```
.anyRequest().authenticated()
```

```
.and()
```

```
- .headers()
```

```
- .frameOptions().disable();
```

```
-
```

```
- http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
```

```
+ .requestMatchers(HttpMethod.GET, "/api/books/**").permitAll()
```

```
+ .requestMatchers("/api/admin/**").hasRole("ADMIN")
```

```
+ .anyRequest().authenticated()
```

```
return http.build();
```

```
}
```

```
@@ -84,9 +84,8 @@
```

```
.anyRequest().authenticated()
```

```
.and()
```

```
.requestMatchers(HttpMethod.GET, "/api/books/**").permitAll()
```

```
- .requestMatchers("/api/admin/**").hasRole("ADMIN")
```

```
- .anyRequest().authenticated()
```

```
-
```

```
+ .requestMatchers("/api/admin/**").hasRole("ADMIN")
```

```
+ .anyRequest().authenticated();
```

```
return http.build();
```

```
}
```

```
@@ -100,17 +100,16 @@
```

```
.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
```

```
.and()
```

```
.authorizeRequests() // authorizeRequests() -> authorizeHttpRequests()
```

```
- .antMatchers("/api/auth/**").permitAll() // antMatchers() removed in Spring 6
```

```
- .antMatchers(HttpMethod.GET, "/api/books/**").permitAll()
```

```

- .antMatchers("/api/admin/**").hasRole("ADMIN")
- .anyRequest().authenticated()
- .and()
- .headers()
- .frameOptions().disable(); // Old H2 console fix

+ .requestMatchers("/api/auth/**").permitAll()
+ .requestMatchers(HttpMethod.GET, "/api/books/**").permitAll()
+ .requestMatchers("/api/admin/**").hasRole("ADMIN")
+ .anyRequest().authenticated()
+ .and()
+ .headers()
+ .frameOptions().disable();

- http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
- }
-

+ http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
+ }

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@@ -42,21 +42,23 @@

@Autowired
private JwtRequestFilter jwtRequestFilter;

- @Override
- protected void configure(AuthenticationManagerBuilder auth) throws Exception {
- // This override approach is removed – must use AuthenticationManagerBuilder differently
- auth.userDetailsService(userDetailsService)
- .passwordEncoder(passwordEncoder());
- }

+ protected void configure(AuthenticationManagerBuilder auth) throws Exception {
+ auth.userDetailsService(userDetailsService) .passwordEncoder(passwordEncoder()); }

@Bean

- @Override

public AuthenticationManager authenticationManagerBean() throws Exception {
- // authenticationManagerBean() removed from WebSecurityConfigurerAdapter in Spring 6
- return authenticationManagerBuilder.getOrBuild();
+ return authenticationManager();
}

@Override

+ public void configure(WebSecurity web) {
+ web.ignoring()
+ .requestMatchers(
+ "/h2-console/**",
+ "/swagger-ui/**",
+ "/v3/api-docs/**",
+ "/webjars/**"

```

```

+ );
+ }

public void configure(WebSecurity web) {
    // WebSecurity.ignoring() is deprecated – use requestMatchers in HttpSecurity
    web.ignoring()
@@ -68,34 +68,28 @@
    "/v3/api-docs/**",
    "/webjars/**"
);

- }
- @Bean
- public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
- http
- .csrf().disable()
- .exceptionHandling()
- .authenticationEntryPoint(jwtAuthEntryPoint)
- .and()
- .sessionManagement()
- .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
- .and()
- .authorizeHttpRequests()
- .requestMatchers("/api/auth/**").permitAll()
- .requestMatchers(HttpMethod.GET, "/api/books/**").permitAll()
- .requestMatchers("/api/admin/**").hasRole("ADMIN")
- .anyRequest().authenticated()
- .and()
- .requestMatchers(HttpMethod.GET, "/api/books/**").permitAll()
- .requestMatchers("/api/admin/**").hasRole("ADMIN")
- .anyRequest().authenticated();
- return http.build();
- }
-
- @Override
- protected void configure(HttpSecurity http) throws Exception {
+ @Bean
+ public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
    .csrf().disable()
    .exceptionHandling()
+ .authenticationEntryPoint(jwtAuthEntryPoint)
+ .and()
+ .sessionManagement()
+ .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
+ .and()
+ .authorizeHttpRequests()
+ .requestMatchers("/api/auth/**").permitAll()

```

```

+ .requestMatchers(HttpMethod.GET, "/api/books/**").permitAll()
+ .requestMatchers("/api/admin/**").hasRole("ADMIN")
+ .anyRequest().authenticated()
+ .and()
+ .headers()
+ .frameOptions().disable();
+
+ http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
+ return http.build();
+ }

```

```

.authenticationEntryPoint(jwtAuthEntryPoint)
.requestMatchers("/api/auth/**").permitAll()
.sessionManagement()

```

@@ -90,9 +90,6 @@

```

http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
}

```

```

- .authenticationEntryPoint(jwtAuthEntryPoint)
- .requestMatchers("/api/auth/**").permitAll()
- .sessionManagement()

```

```

.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
.and()
.authorizeRequests() // authorizeRequests() -> authorizeHttpRequests()

```

@@ -90,7 +90,6 @@

```

http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
}

```

```

- .sessionCreationPolicy(SessionCreationPolicy.STATELESS)

```

```

.and()
.authorizeRequests() // authorizeRequests() -> authorizeHttpRequests()
.requestMatchers("/api/auth/**").permitAll()

```

@@ -30,7 +30,7 @@

```

*/

```

```

@Configuration

```

```

@EnableWebSecurity

```

```

-@EnableGlobalMethodSecurity(prePostEnabled = true, securedEnabled = true) // Deprecated: use

```

```

@EnableMethodSecurity

```

```

+ @EnableMethodSecurity(prePostEnabled = true, securedEnabled = true) // Use @EnableMethodSecurity
instead of @EnableGlobalMethodSecurity

```

```

public class SecurityConfig {

```

```

@Autowired

```

@@ -90,13 +90,6 @@

```

http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
}

```

```

- .and()

```

```

- .authorizeRequests() // authorizeRequests() -> authorizeHttpRequests()
- .requestMatchers("/api/auth/**").permitAll()
- .requestMatchers(HttpMethod.GET, "/api/books/**").permitAll()
- .requestMatchers("/api/admin/**").hasRole("ADMIN")
- .anyRequest().authenticated()
- .and()

.headers()
.frameOptions().disable();
@@ -61,13 +61,14 @@
}

public void configure(WebSecurity web) {
// WebSecurity.ignoring() is deprecated – use requestMatchers in HttpSecurity

- web.ignoring()
- .requestMatchers(
- "/h2-console/**",
- "/swagger-ui/**",
- "/v3/api-docs/**",
- "/webjars/**"
- );
+ web.ignoring()
+ .requestMatchers(
+ "/h2-console/**",
+ "/swagger-ui/**",
+ "/v3/api-docs/**",
+ "/webjars/**"
+ );
+ }

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
http
@@ -91,11 +91,13 @@
http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
}

- .headers()
- .frameOptions().disable();
+ .headers()
+ .frameOptions().disable()
+ .and()
+ .addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
+ return http.build();
+ }

- http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
- }

@Bean
public PasswordEncoder passwordEncoder() {

```

```
return new BCryptPasswordEncoder();
```

```
@@ -102,4 +102,6 @@
```

```
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}
```

```
+ return new BCryptPasswordEncoder();  
+ }  
}
```

```
@@ -94,7 +94,9 @@
```

```
.headers()  
.frameOptions().disable()  
.and()
```

```
- .addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);  
+ .addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class)  
+ .and()  
+ .build();
```

```
return http.build();  
}
```

```
@@ -102,6 +104,5 @@
```

```
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}
```

```
- return new BCryptPasswordEncoder();  
- }  
}
```

```
+
```

```
@@ -101,8 +101,10 @@
```

```
}
```

```
@Bean
```

```
+ @Bean
```

```
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}  
}  
}
```

```
+
```

```
@@ -101,10 +101,10 @@
```

```
}
```

```
@Bean
```

```
- @Bean
```

```
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}  
}  
}
```



```

}
+}
@@ -42,11 +42,9 @@
@Autowired
private JwtRequestFilter jwtRequestFilter;

- protected void configure(AuthenticationManagerBuilder auth) throws Exception {
auth.userDetailsService(userDetailsService) .passwordEncoder(passwordEncoder()); }
-

@Bean
- public AuthenticationManager authenticationManagerBean() throws Exception {
- return authenticationManager();
+ public AuthenticationManager authenticationManager(AuthenticationConfiguration
authenticationConfiguration) throws Exception {
+ return authenticationConfiguration.getAuthenticationManager();
}

@Override
@@ -106,3 +106,4 @@
}
}
}

+}
@@ -49,15 +49,6 @@
@Override
public void configure(WebSecurity web) {

- web.ignoring()
- .requestMatchers(
- "/h2-console/**",
- "/swagger-ui/**",
- "/v3/api-docs/**",
- "/webjars/**"
- );
- }
- public void configure(WebSecurity web) {
// WebSecurity.ignoring() is deprecated – use requestMatchers in HttpSecurity
web.ignoring()
.requestMatchers(
@@ -98,3 +98,4 @@
}
}
}

+}
@@ -93,9 +93,4 @@
public PasswordEncoder passwordEncoder() {
return new BCryptPasswordEncoder();
}

-}

```

```
-}  
-}  
-}  
-}  
-}
```

```
+}}
```

```
@@ -93,4 +93,5 @@
```

```
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}
```

```
+
```

```
}}
```

```
@@ -94,4 +94,5 @@
```

```
return new BCryptPasswordEncoder();  
}
```

```
+
```

```
}}
```

```
@@ -95,4 +95,5 @@
```

```
}
```

```
+
```

```
}}
```

```
@@ -96,4 +96,5 @@
```

```
+
```

```
}}
```

```
■ src/main/java/com/library/service/BorrowService.java
```

```
@@ -97,6 +97,27 @@
```

```
record.getBook().getTitle(), daysOverdue, record.getFineAmount());  
}
```

```
+ logger.info("Book '{}' borrowed by user '{}', due: {}", book.getTitle(), username,  
record.getDueDate());  
+ return modelMapper.map(saved, BorrowRecordDto.class);  
+ }  
+  
+ public BorrowRecordDto returnBook(Long recordId, String username) {  
+ BorrowRecord record = borrowRecordRepository.findById(recordId)  
+ .orElseThrow(() -> new BookNotFoundException("Borrow record not found: " + recordId));  
+ if (!record.getUser().getUsername().equals(username)) {  
+ throw new SecurityException("This borrow record doesn't belong to user: " + username);  
+ }  
+ record.setReturnDate(LocalDate.now());  
+ record.setReturned(true);  
+ record.setStatus(BorrowRecord.BorrowStatus.RETURNED);  
+ }
```

```

+ if (LocalDate.now().isAfter(record.getDueDate())) {
+ long daysOverdue = record.getDueDate().until(LocalDate.now()).getDays();
+ record.setFineAmount(daysOverdue * FINE_PER_DAY);
+ logger.warn("Book '{}' returned {} days overdue. Fine: ${}",
+ record.getBook().getTitle(), daysOverdue, record.getFineAmount());
+ }
+

```

```

bookRepository.incrementAvailableCopies(record.getBook().getId());
BorrowRecord saved = borrowRecordRepository.save(record);
return modelMapper.map(saved, BorrowRecordDto.class);

```

@@ -124,3 +145,8 @@

```

borrowRecordRepository.saveAll(updated);
}
}

```

```

+ }).collect(Collectors.toList());
+
+ borrowRecordRepository.saveAll(updated);
+ }
+ }

```

@@ -136,17 +136,14 @@

```

List<BorrowRecord> overdueRecords = borrowRecordRepository.findOverdueRecords(LocalDate.now());
logger.info("Found {} overdue records", overdueRecords.size());

```

```

- List<BorrowRecord> updated = overdueRecords.stream().peek(record -> {
- record.setStatus(BorrowRecord.BorrowStatus.OVERDUE);
- long daysOverdue = record.getDueDate().until(LocalDate.now()).getDays();
- record.setFineAmount(daysOverdue * FINE_PER_DAY);
- }).collect(Collectors.toList());

```

```

+ List<BorrowRecord> updated = overdueRecords.stream()
+ .peek(record -> {
+ record.setStatus(BorrowRecord.BorrowStatus.OVERDUE);
+ long daysOverdue = record.getDueDate().until(LocalDate.now()).getDays();
+ record.setFineAmount(daysOverdue * FINE_PER_DAY);
+ })
+ .collect(Collectors.toList());

```

```

borrowRecordRepository.saveAll(updated);
}
}

```

```

- }).collect(Collectors.toList());
-
- borrowRecordRepository.saveAll(updated);
- }
- }

```